

[8] Project & CNNs



Design Project

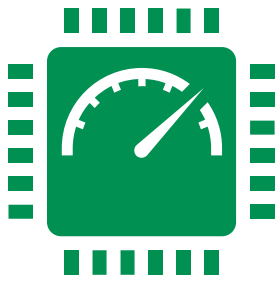
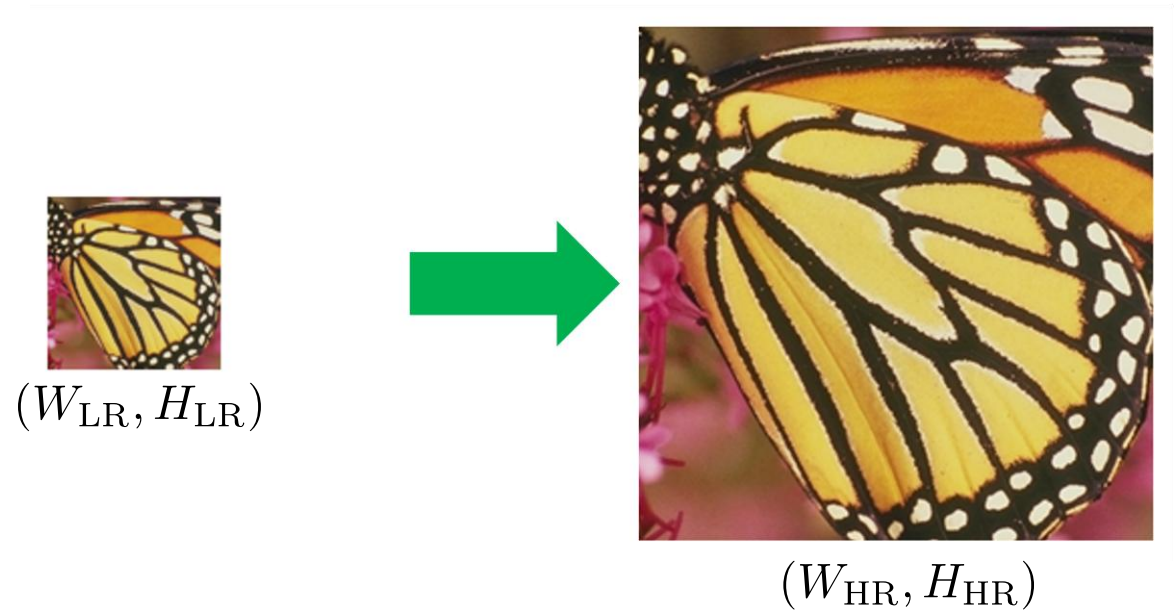


Image super-resolution: reconstruct high resolution image from one or more low resolution images

- Long-standing research problem with applications for satellite, medical, and sensor imaging

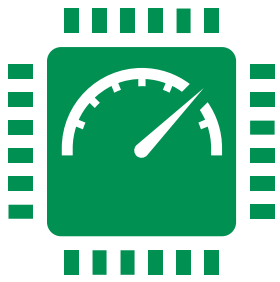
As with many computer vision tasks, deep learning ML approaches have been primary focus of recent years

- ELEN90096 Project is to accelerate SRCNN algorithm of [1] in HLS and deploy to Kria KV260



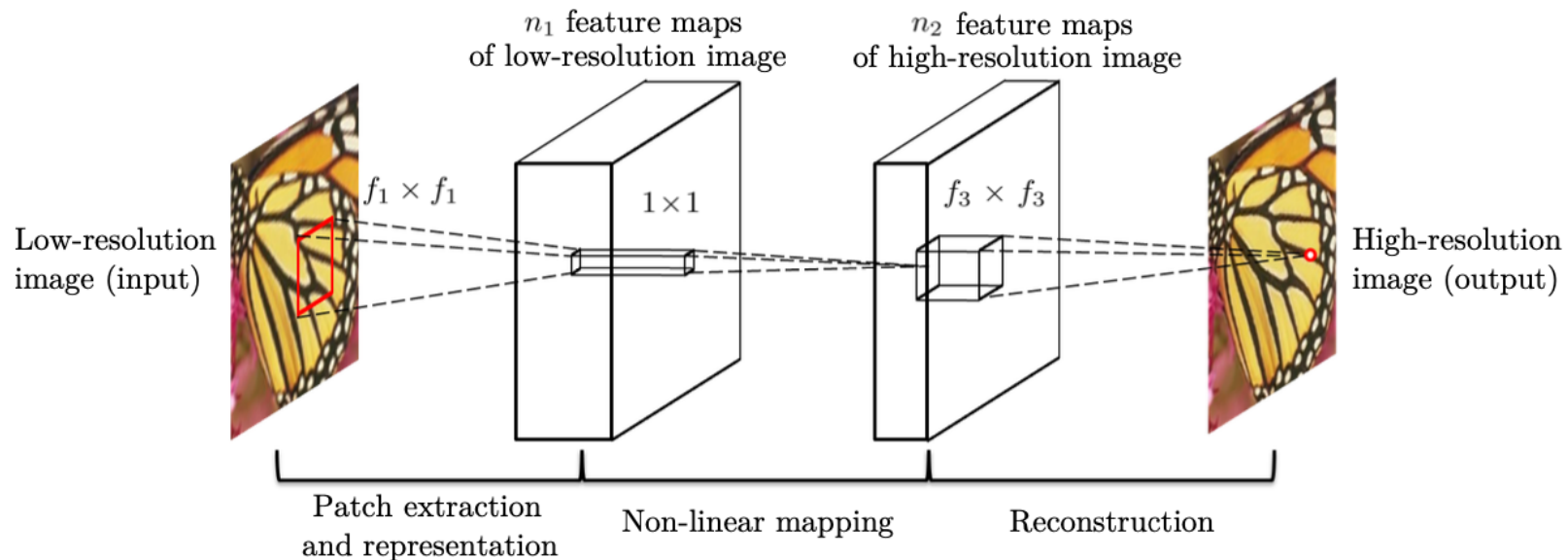
[1] C. Dong et al., "Learning a deep convolutional network for image super-resolution," Computer Vision–ECCV 2014.

Design Project

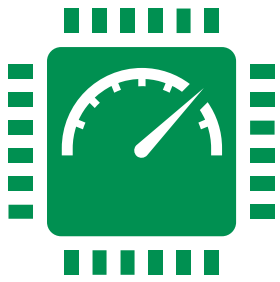


SRCNN is a 3-layer *convolutional neural network*

- Each layer is a computationally intensive, multidimensional calculation
⇒ parallelization
- Multiple layers that must be executed with significant memory access
⇒ dataflow



Design Project



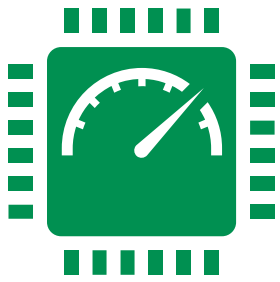
Major project milestones:

1. **Golden Reference** code submission – functionally correct, baseline C/C++ sequential implementation of SRCNN
2. **Status Update** – oral presentation to discuss approach, progress, and remaining tasks
3. **Final Report** – complete documentation of your solution and its performance, along with HLS and PYNQ host code submission

Deliverable	Due	Subject Weight
Golden Reference	5 Oct	5%
Status Update	12 Oct	10%
Final Report	19 Oct*	25%

See Canvas assignments for additional details on what is expected for each milestone

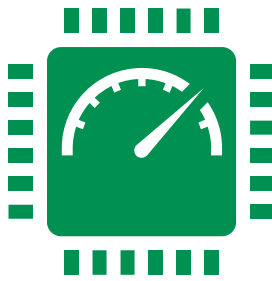
Remaining Syllabus



Week	Lecture	Project
7	CNNs & Block Matrix Example	
8	Mid-Semester Test	
9	CNN Optimizations	Start Project
<i>Mid-Semester Break</i>		
10	GPU Architectures and Cloud	Golden Reference
11	Review	Project Update
12		Final Report*

Convolutional Neural Networks (CNNs)

Machine Learning

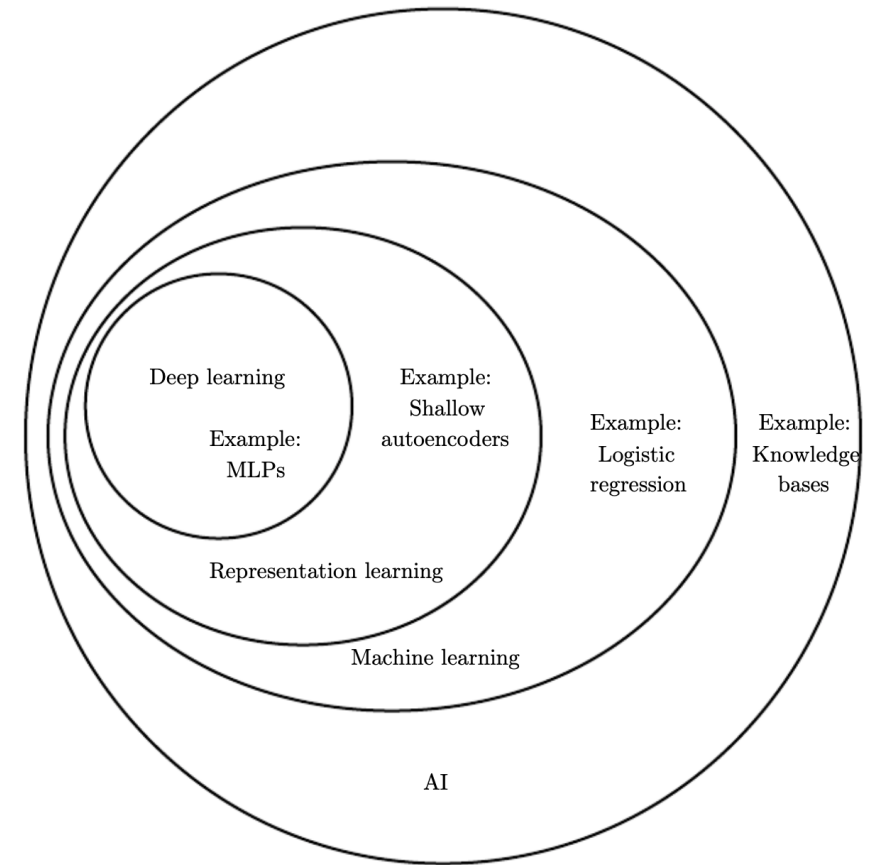


Machine learning is the ability to extract patterns from raw data

- Performance at some task T , as measured by metric P , improves with experience E
- *Mitchell, 1997*

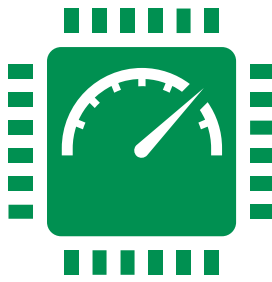
- *Tasks*: classification, regression, denoising, ...
- *Experience*: datasets - images, sound clips, databases, ...
- *Performance*: accuracy, error rate, ...

Supervised learning – examples in dataset include label or target, i.e., the correct output/decision



Goodfellow et al., Deep Learning
<http://www.deeplearningbook.org>

Linear Regression



Task: predict value $y \in \mathbb{R}$ given input $x \in \mathbb{R}^n$

Experience: $(\mathbf{X}^{(train)}, \mathbf{y}^{(train)})$

Performance: Mean Squared Error

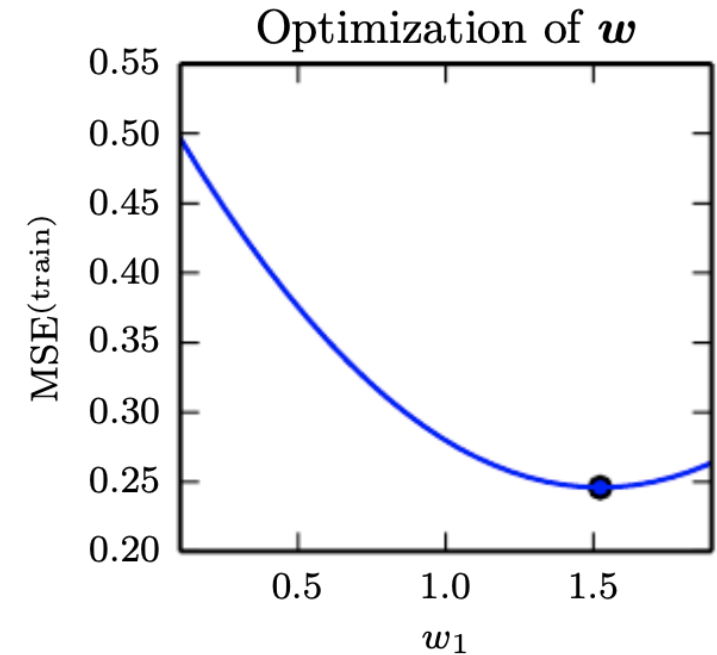
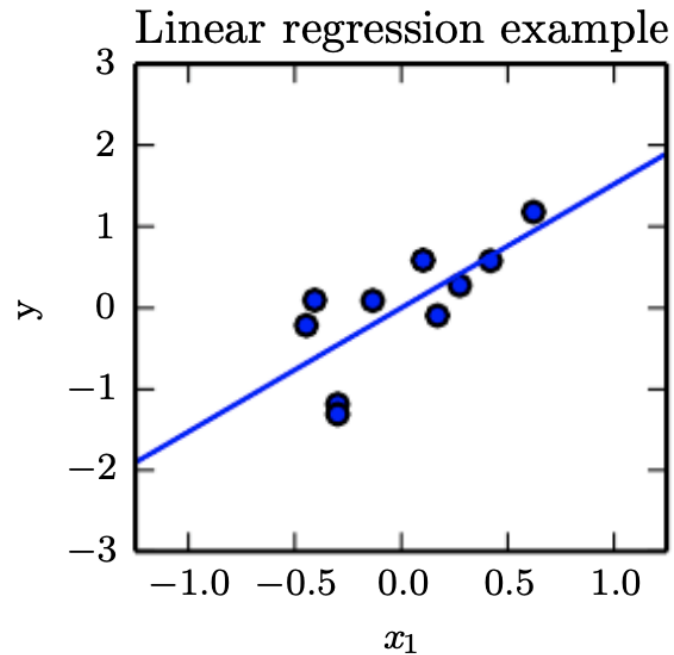
$$\text{MSE}_{\text{test}} = \frac{1}{m} \|\hat{\mathbf{y}}^{(\text{test})} - \mathbf{y}^{(\text{test})}\|_2^2$$

Algorithm: Linear Regression with parameter weights $\mathbf{w} \in \mathbb{R}^n$

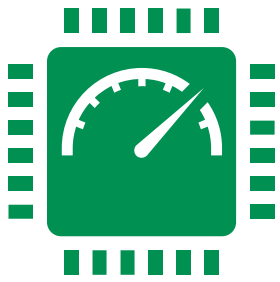
$$\hat{y} = \mathbf{w}^T \mathbf{x} = \sum_i w_i \cdot x_i$$

Learning:

$$\nabla_{\mathbf{w}} \frac{1}{m} \|\hat{\mathbf{y}}^{(\text{train})} - \mathbf{y}^{(\text{train})}\|_2^2 = 0$$



[1] Goodfellow et al., Deep Learning <http://www.deeplearningbook.org>



Logistic Regression

Sigmoidal logistic function applied to the output of linear (affine) regression

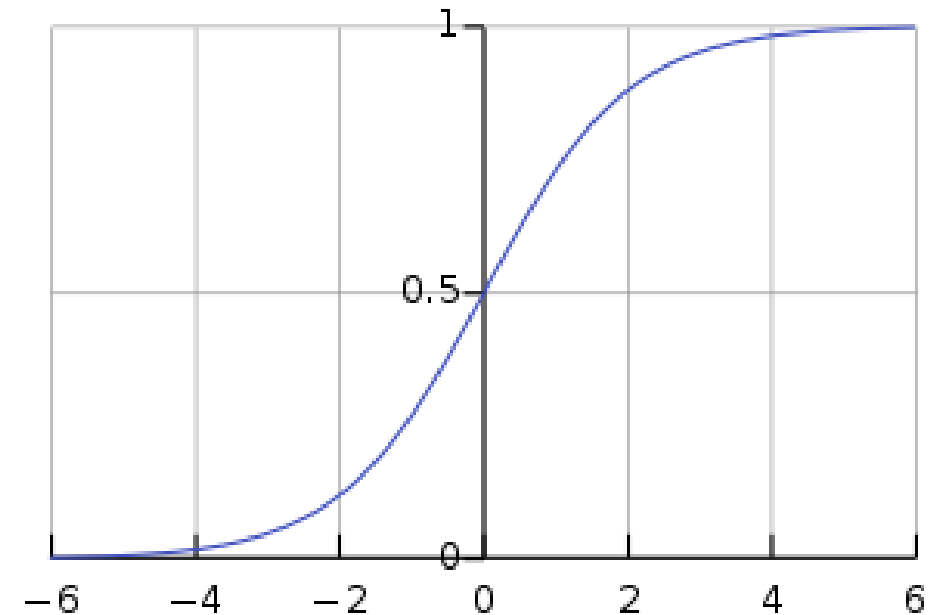
$$\hat{y} = g(w^T \mathbf{x}) = g\left(\sum_i w_i \cdot x_i + b\right)$$

Non-linearity allows universal function approximation

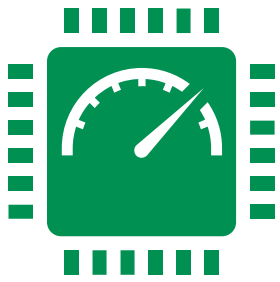
- Numerical optimization in learning as no longer a closed form solution

$$\nabla_w \frac{1}{m} \|\hat{\mathbf{y}}^{(\text{train})} - \mathbf{y}^{(\text{train})}\|_2^2 = 0$$

https://en.wikipedia.org/wiki/Logistic_regression

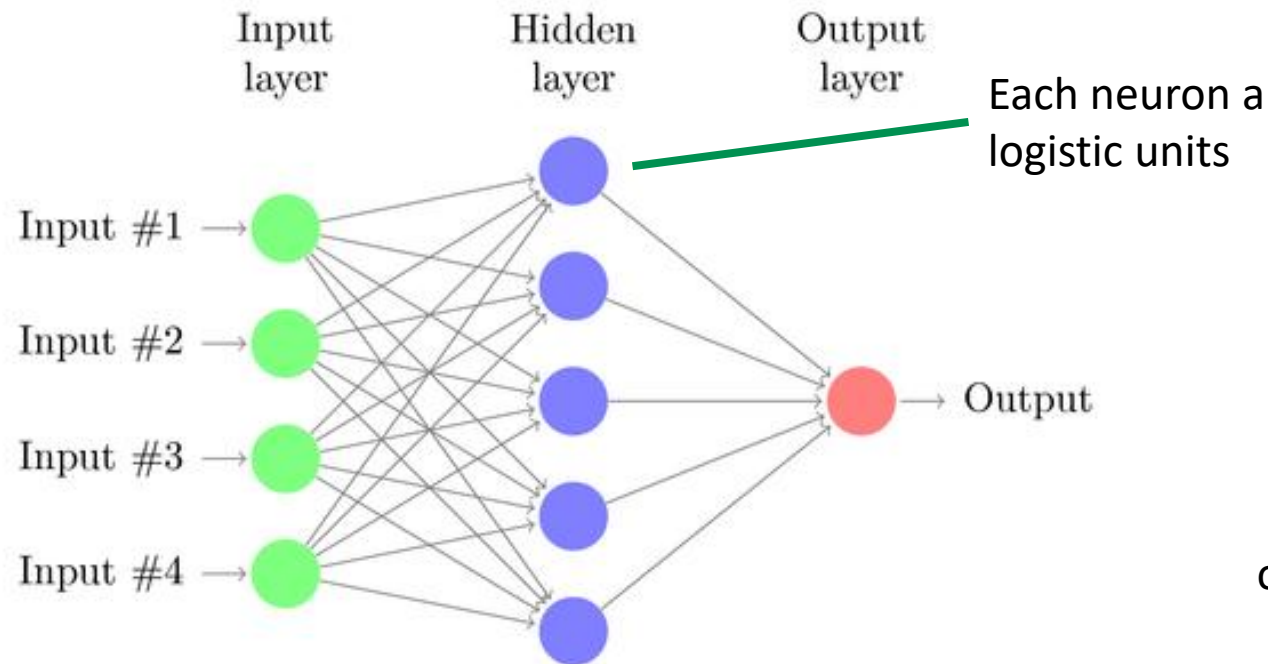


$$g(z) = \frac{e^z}{e^z + 1}$$



Multilayer Perceptron (MLP)

Feedforward neural network architecture



$$x_i = g_i(W_i * x_{i-1} + b_i)$$

activation function

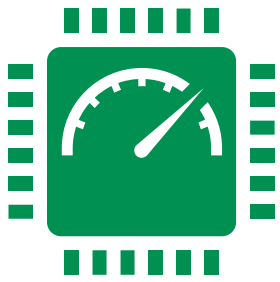
input features $n_{i-1} \times 1$

output features $n_i \times 1$

weight matrix $n_i \times n_{i-1}$

bias vector $n_i \times 1$

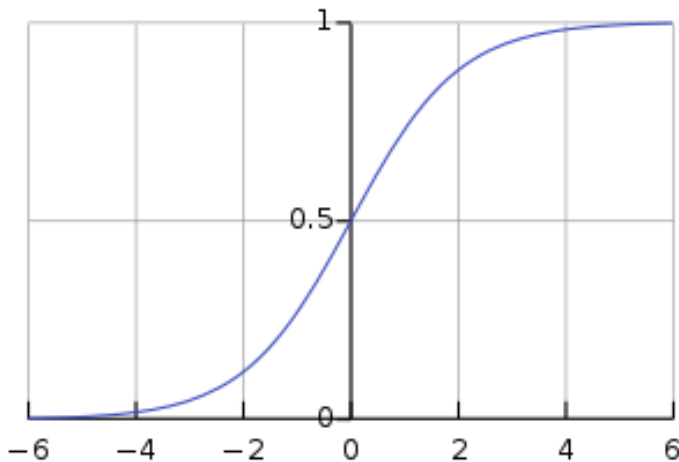
<https://towardsdatascience.com/simple-introduction-to-convolutional-neural-networks-cdf8d3077bac>



Activation Function

Sigmoid

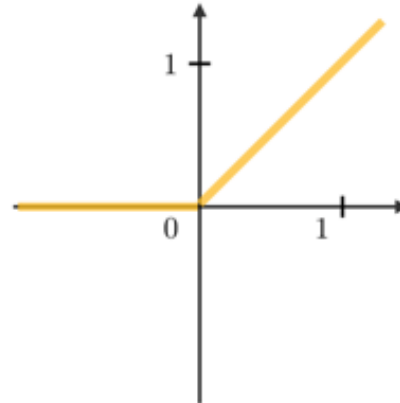
$$g(z) = \frac{e^z}{e^z + 1}$$



https://en.wikipedia.org/wiki/Logistic_regression

ReLU

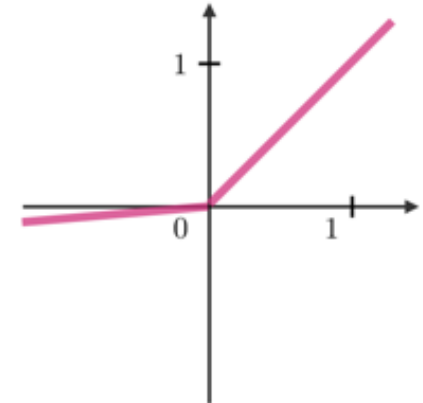
$$g(z) = \max(0, z)$$



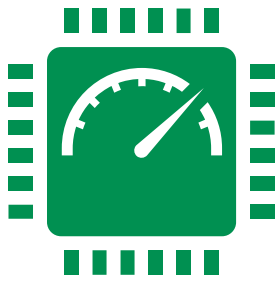
Leaky ReLU

$$g(z) = \max(\epsilon z, z)$$

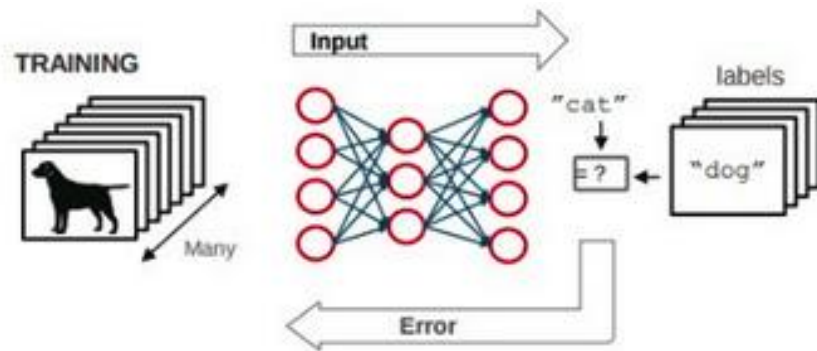
with $\epsilon \ll 1$



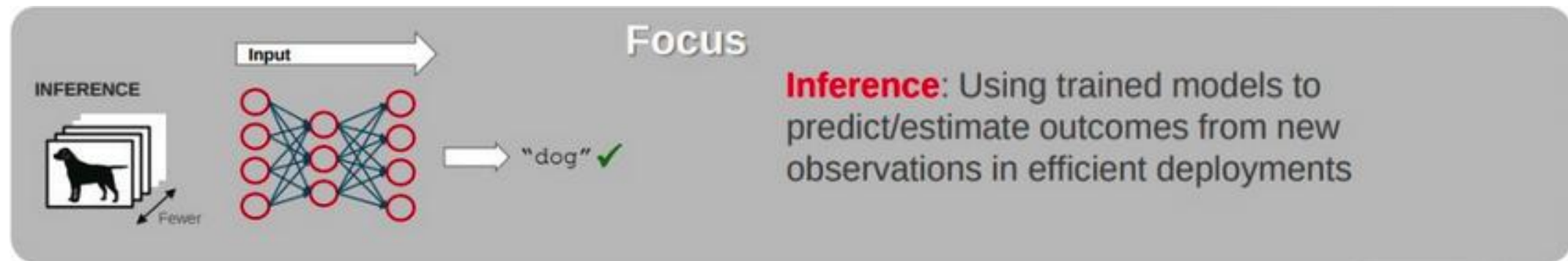
[4] A. Amidi and S. Amidi, "CNN Cheatsheet," <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>



Training versus Inference



Training: Process for machine to “learn” and optimize model from data



Inference: Using trained models to predict/estimate outcomes from new observations in efficient deployments

Deep Learning

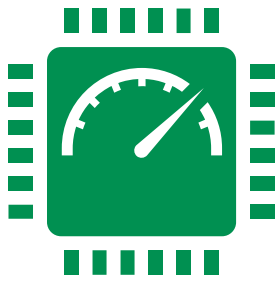
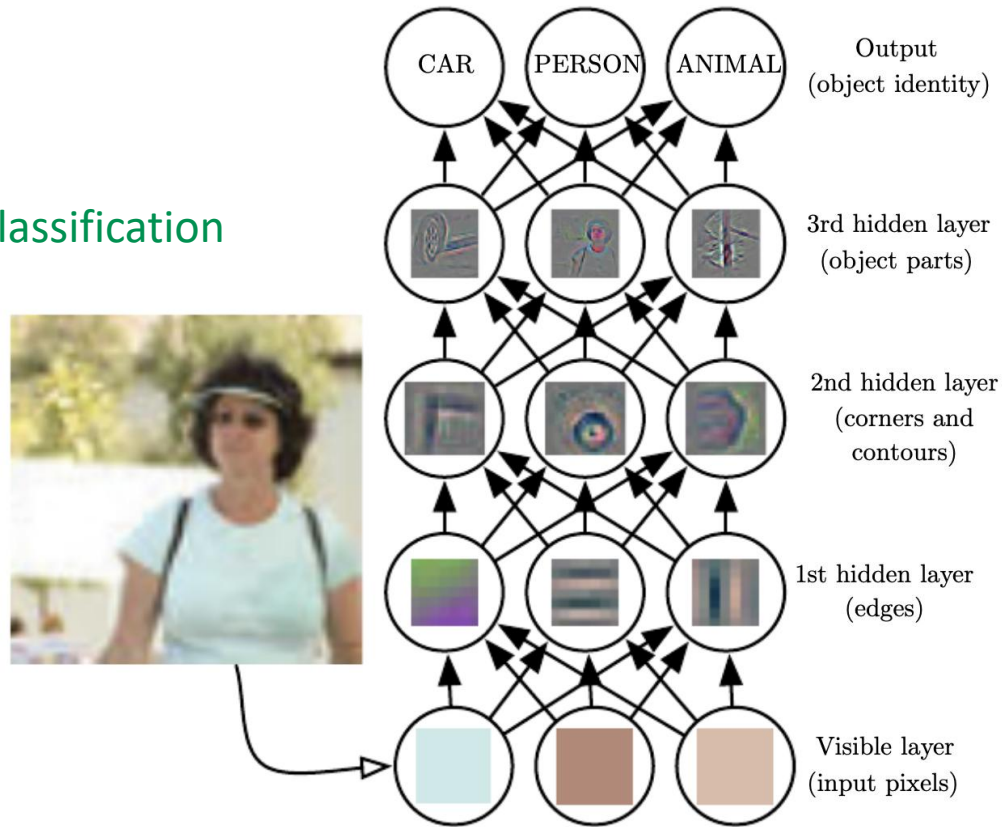
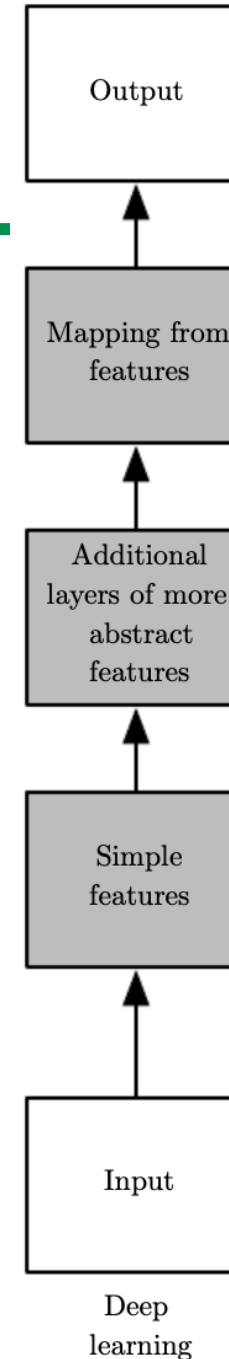


Image classification



Goodfellow et al., Deep Learning
<http://www.deeplearningbook.org>



Deep
learning

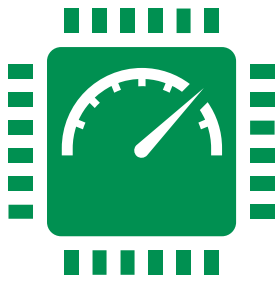


Image Representations

Raster graphics represent image as 2D rectangular matrix of *pixels*

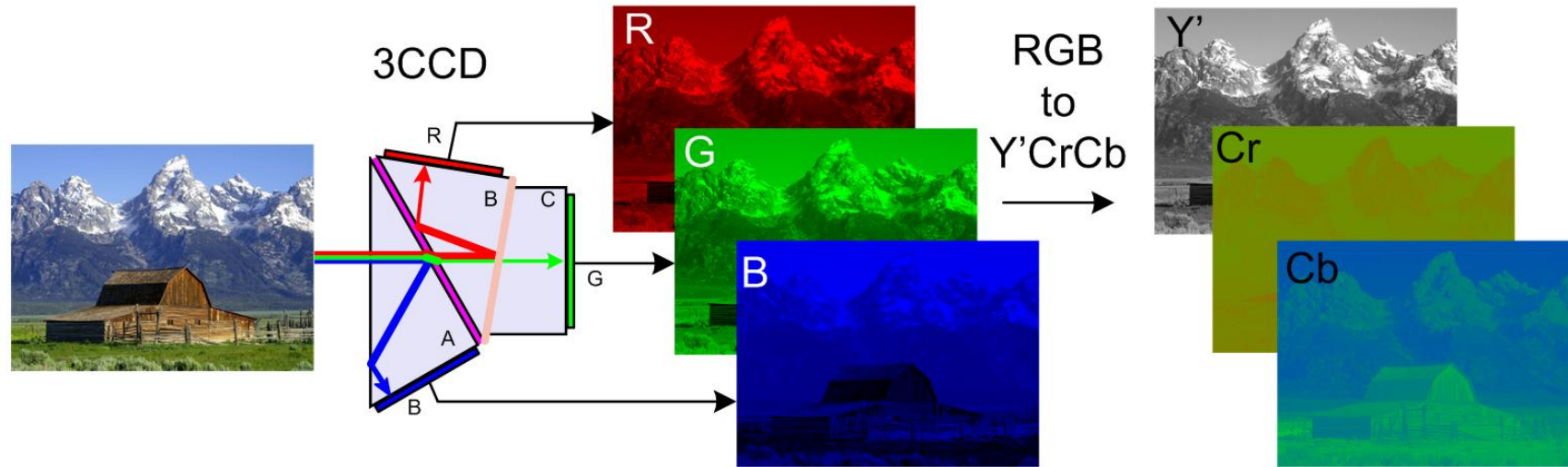
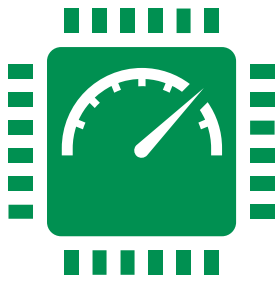
- width x height
- bits per pixel
- Possibly multiple channels
 - Colour space, e.g., RGB
 - Image + depth



https://en.wikipedia.org/wiki/Raster_graphics

<https://en.wikipedia.org/wiki/YCbCr>

Colour Spaces



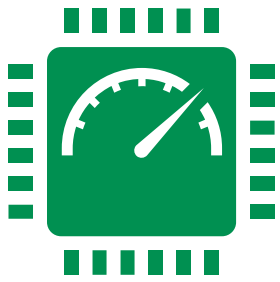
<https://en.wikipedia.org/wiki/YCbCr>

RGB is not only possible colour space

- Originally to drive cathode ray tubes
- Redundant information captured by the 3 planes

Y'CbCr is another commonly used colour space

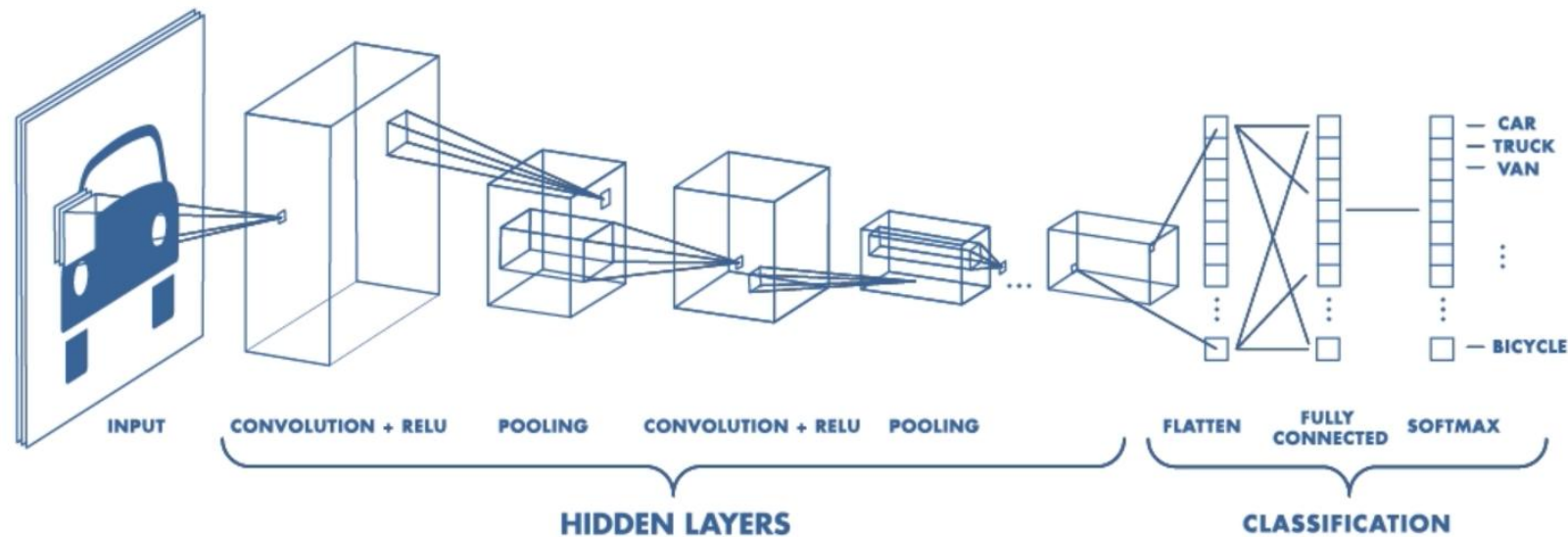
- Luma plane Y' – brightness
- Chroma planes Cb and Cr – blue and red differences
- Chroma planes can be represented with lower resolution as human colour perception is less



Convolutional Neural Networks

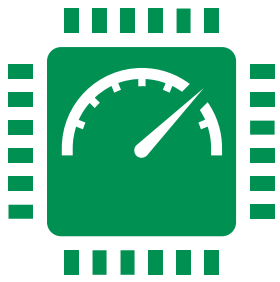
CNNs a form of DNNs where simplified connectivity is used to manage parameters to be learned

- Convolutional layers with possible non-linear activations – feature extraction and representation
- Pooling layers help reduce complexity of representation
- Fully-connected layers at end of network to make decision based on extracted features



<https://au.mathworks.com/discovery/convolutional-neural-network-matlab.html>

Convolutional Layer



The basic convolutional layer implements a 2D discrete convolution between a filter kernel and an input feature map

Filter kernel
 $f_i \times f_i$

0	1	2
2	2	0
0	1	2

Input feature map
 $h_{i-1} \times w_{i-1}$

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

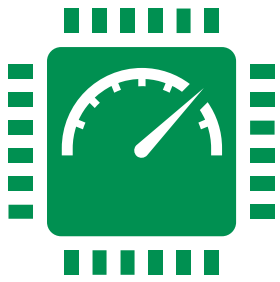
Output feature map
 $h_i \times w_i$

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

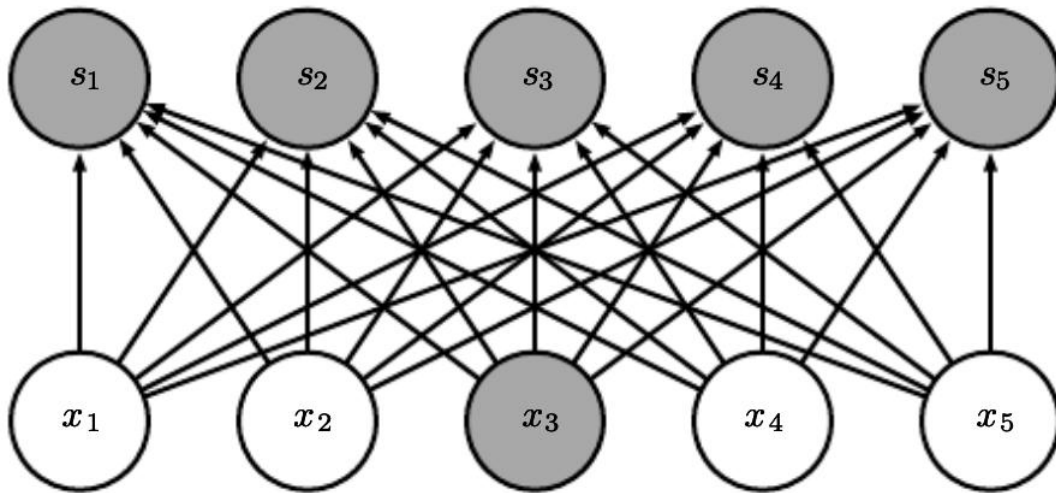
$*$ $=$

[3] Vincent Dumoulin, Francesco Visin - [A guide to convolution arithmetic for deep learning](#)

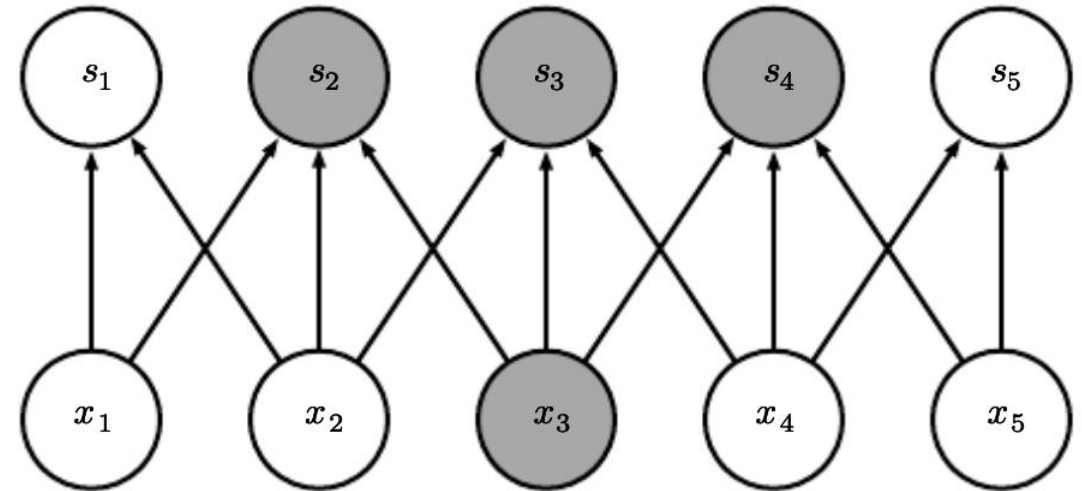
Convolutional Layers



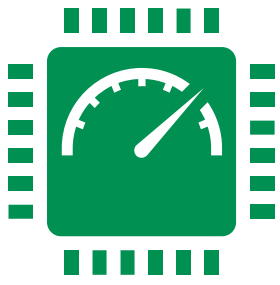
Dense connectivity



Sparse connectivity
(with parameter reuse)



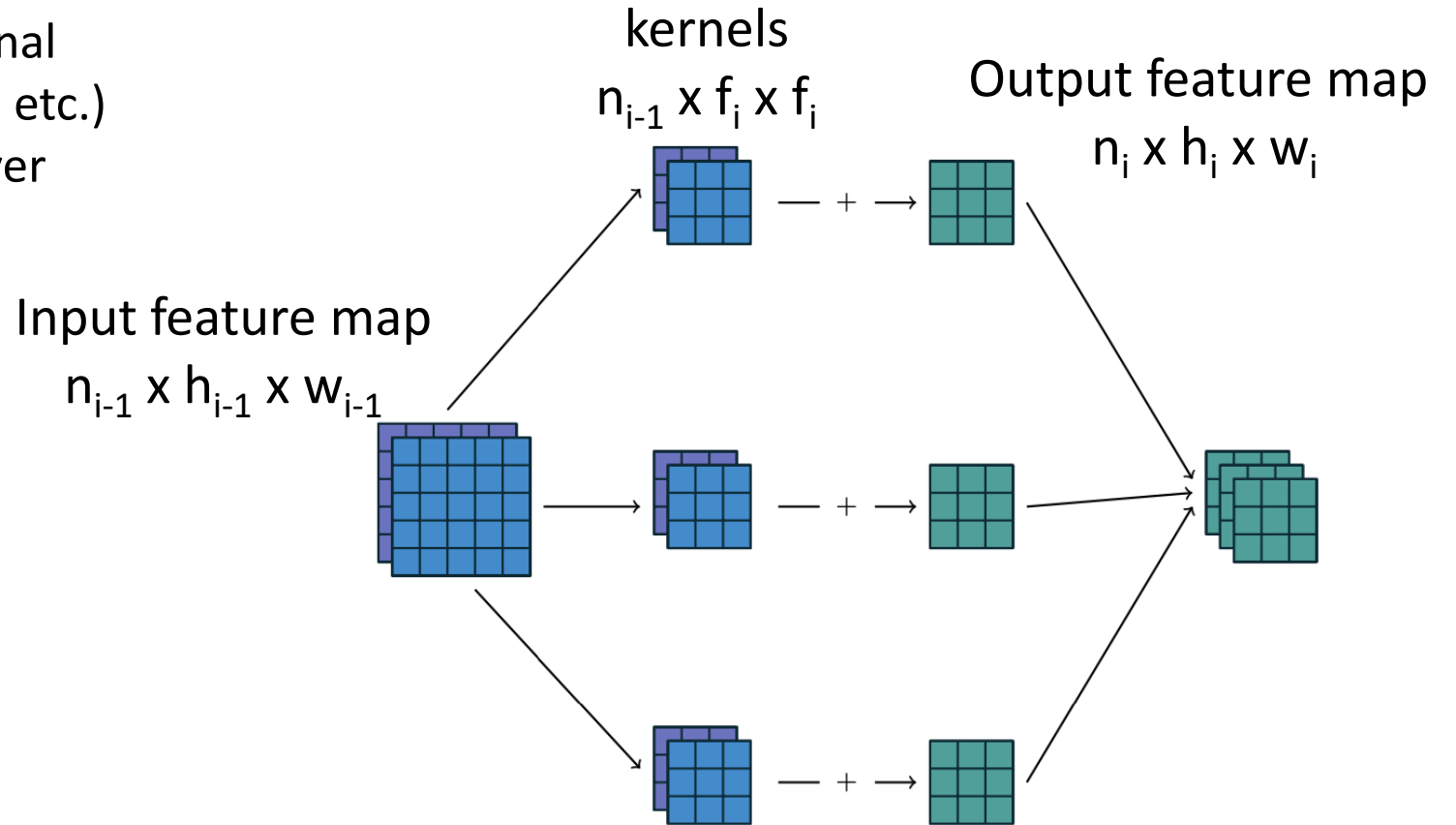
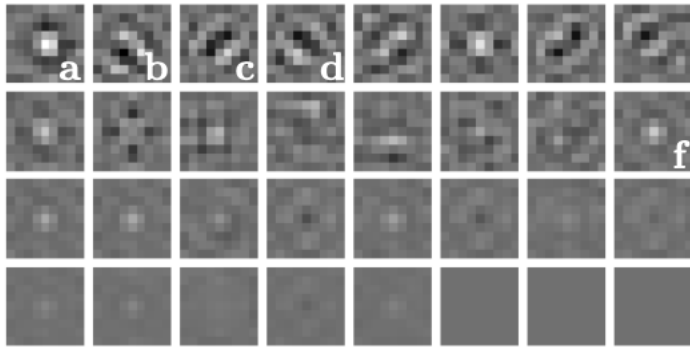
[1] Goodfellow et al., Deep Learning <http://www.deeplearningbook.org>



Additional Feature Dimensions

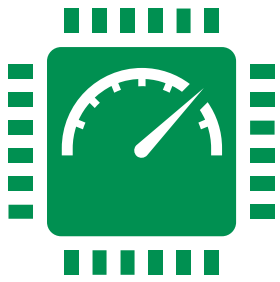
Feature maps typically three dimensional

- Multichannel inputs (RGB or Y'CbCr, etc.)
- Multiple output features at each layer

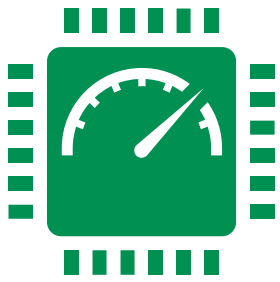


[3] Vincent Dumoulin, Francesco Visin - [A guide to convolution arithmetic for deep learning](#)

Complexity of CNNs



```
for (n = 0; n < N; n++) { // output feature
    for (h = 0; h < H; h++) { // output height
        for (w = 0; w < W; w++) { // output width
            // kernel computation
            for (i = 0; i < F; i++) { // kernel height
                for (j = 0; j < F; j++) { // kernel width
                    for (k = 0; k < N_in; k++) { // input feature
                        out[n][h][w] += W[k][i][j] * in[k][h+i][w+j];
                    }
                }
            }
        }
        out[n][w][h] = f(out[n][w][h] + b[n]); // bias and activation
    }
}
```



Input Padding

Without input *padding*, width and height of the feature map shrinks at each layer

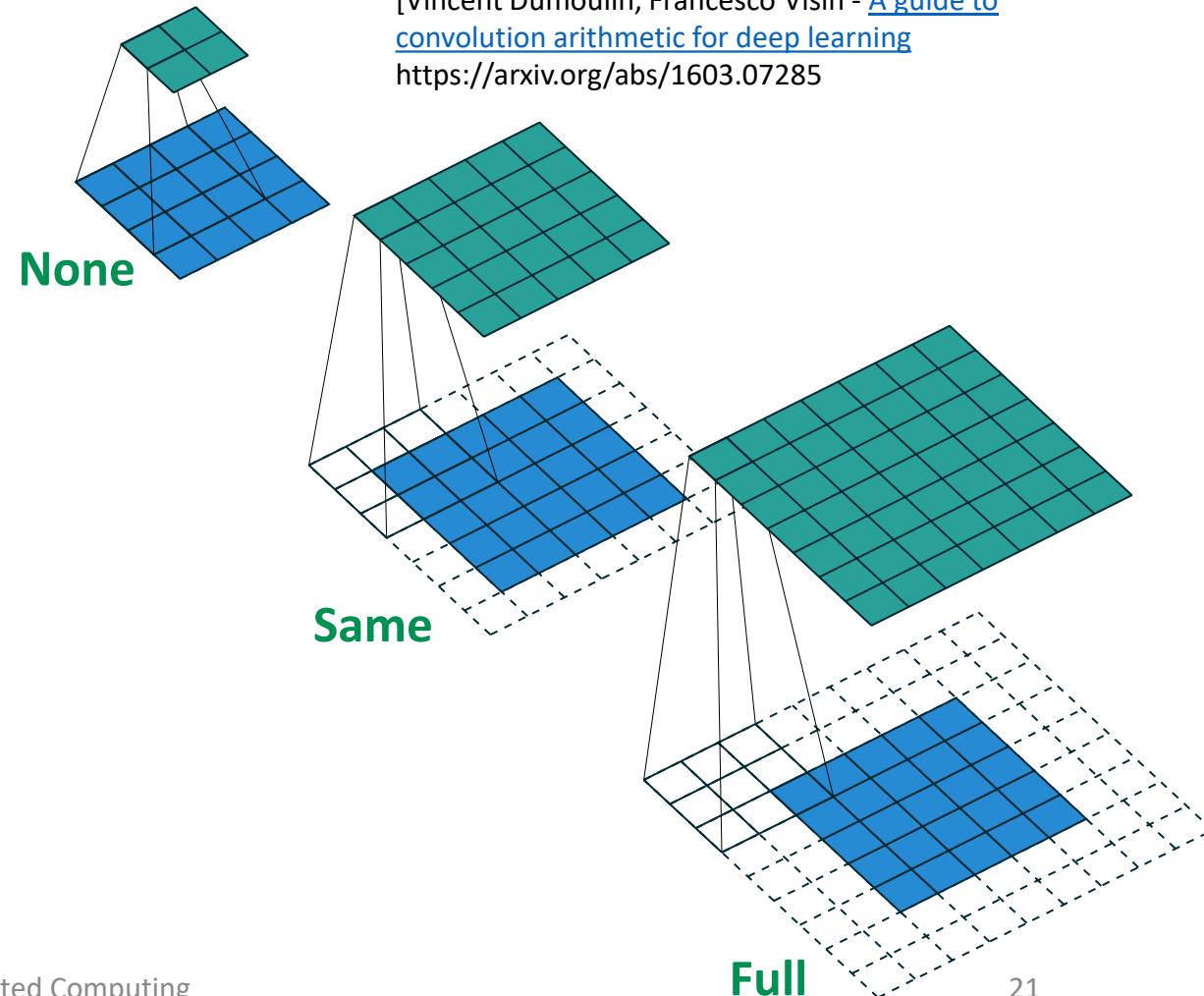
- Padding can be applied using either zeros or by extending edge values.

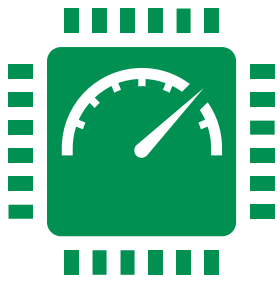
Common choices include

- None – width/height shrink
- Same – width/height constant
- Full – width/height grow

Edge cases must be handled in computation

[Vincent Dumoulin, Francesco Visin - [A guide to convolution arithmetic for deep learning](https://arxiv.org/abs/1603.07285)
<https://arxiv.org/abs/1603.07285>

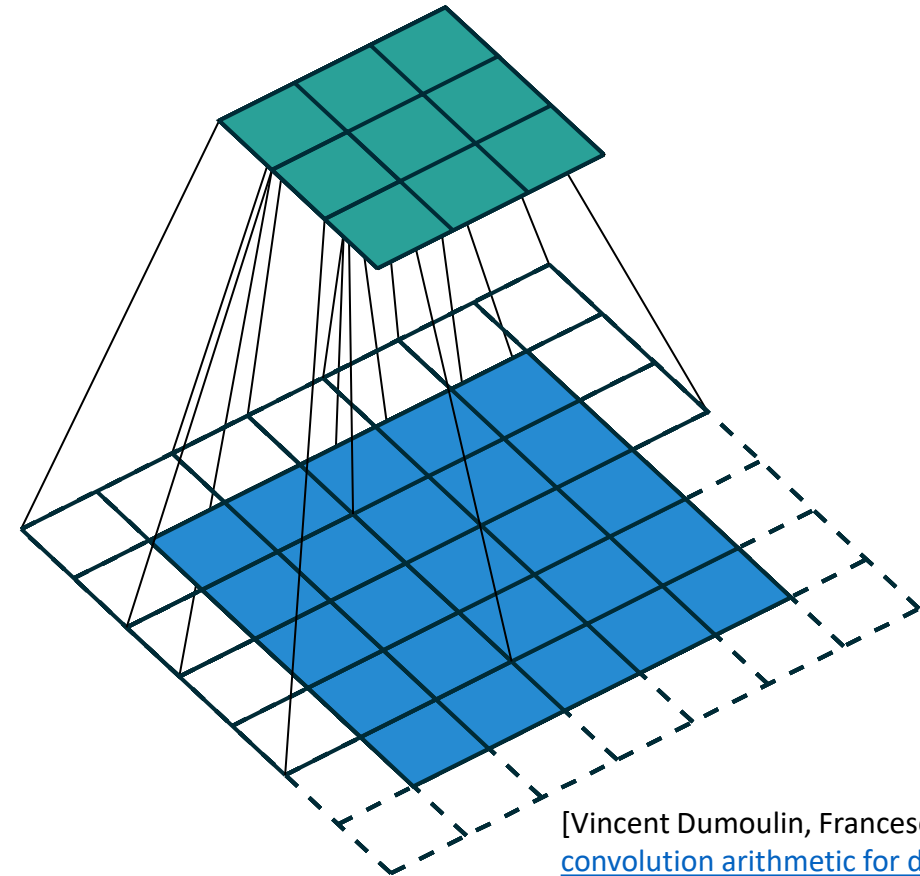




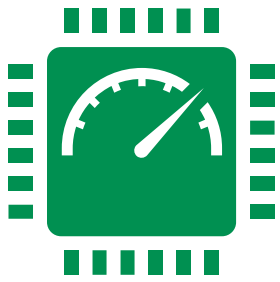
Stride

Common to use a non-unit *stride* when shifting kernel function through input feature map

- Reduces output size
- Stride offset must also be handled in computation



[Vincent Dumoulin, Francesco Visin - [A guide to convolution arithmetic for deep learning](https://arxiv.org/abs/1603.07285)
<https://arxiv.org/abs/1603.07285>



Pooling Layer

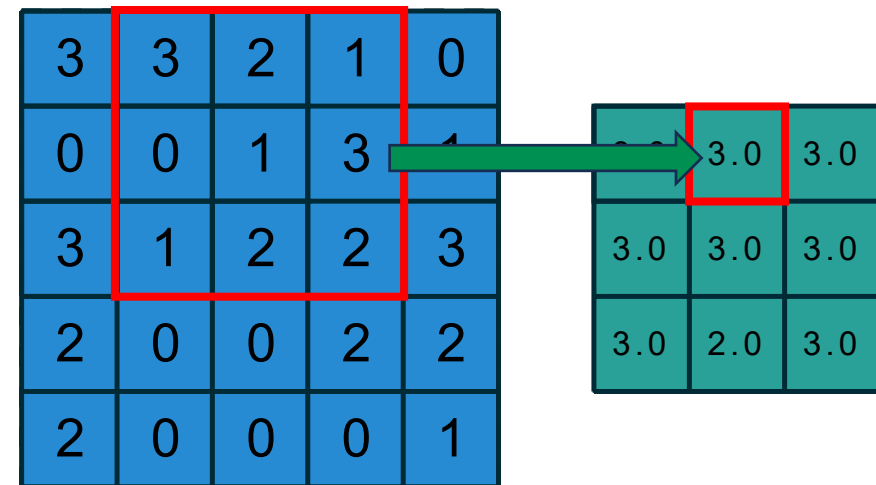
Pooling layers used to further reduce size of feature map

- Max pooling (most common)
- Average pooling

Like convolutional layer has a

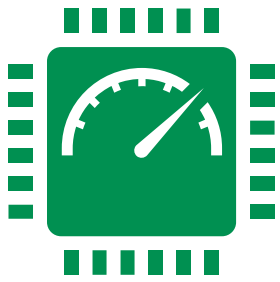
- Kernel size
- Stride

MAX Pooling Example



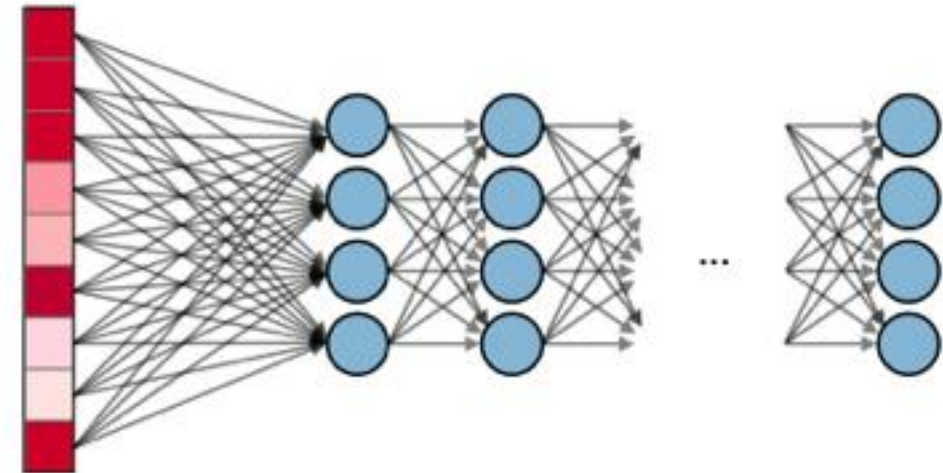
[Vincent Dumoulin, Francesco Visin - [A guide to convolution arithmetic for deep learning](https://arxiv.org/abs/1603.07285)
<https://arxiv.org/abs/1603.07285>

Fully Connected Layers

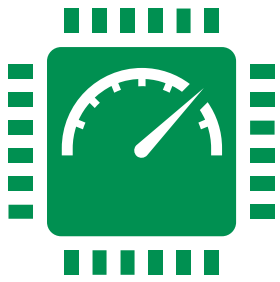


Fully connected layers occur at the end of network

- Unlike convolutional layers, dense connections and no parameter sharing
- Useful to map to task objective, e.g., classification

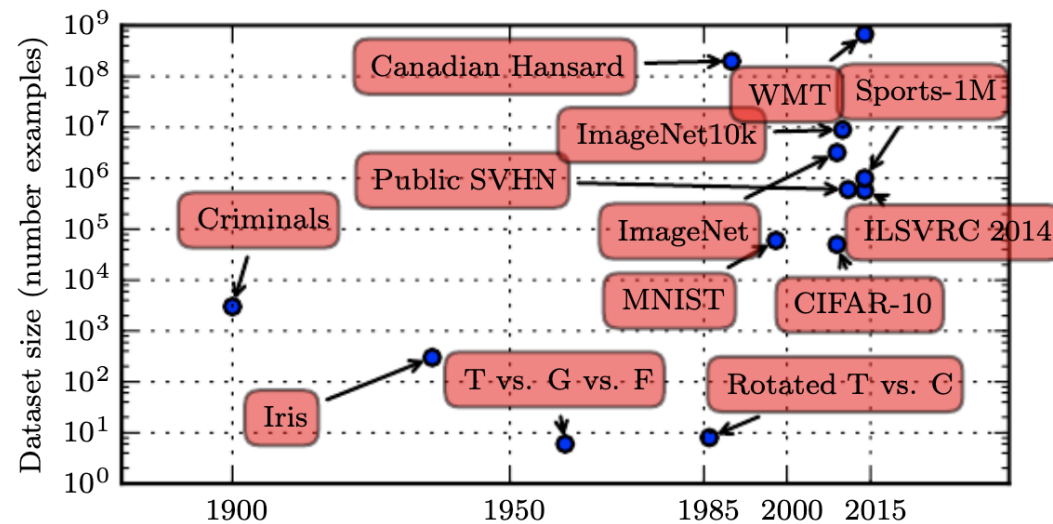


[4] A. Amidi and S. Amidi, "CNN Cheatsheet," <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>



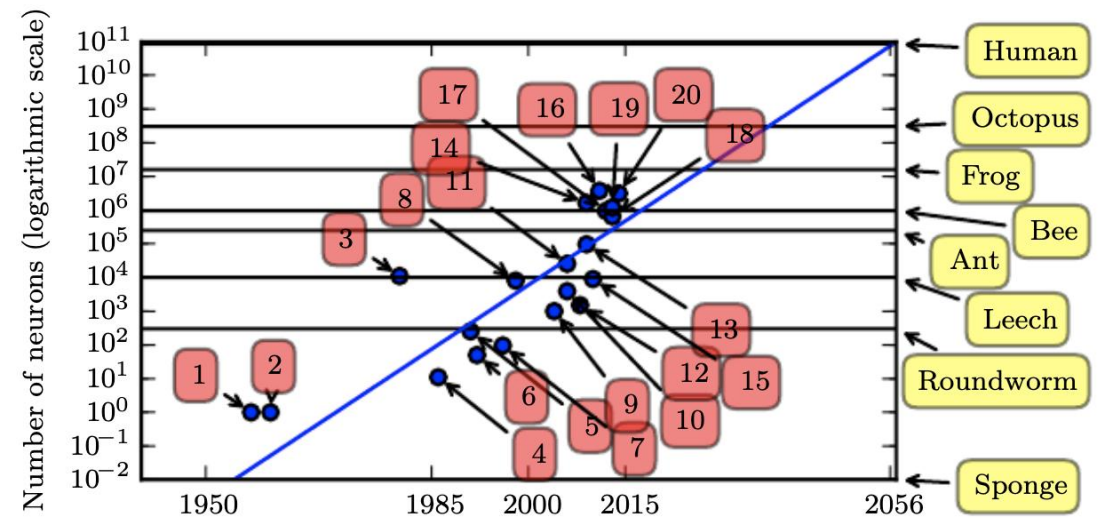
The Need for Acceleration

Dataset Size



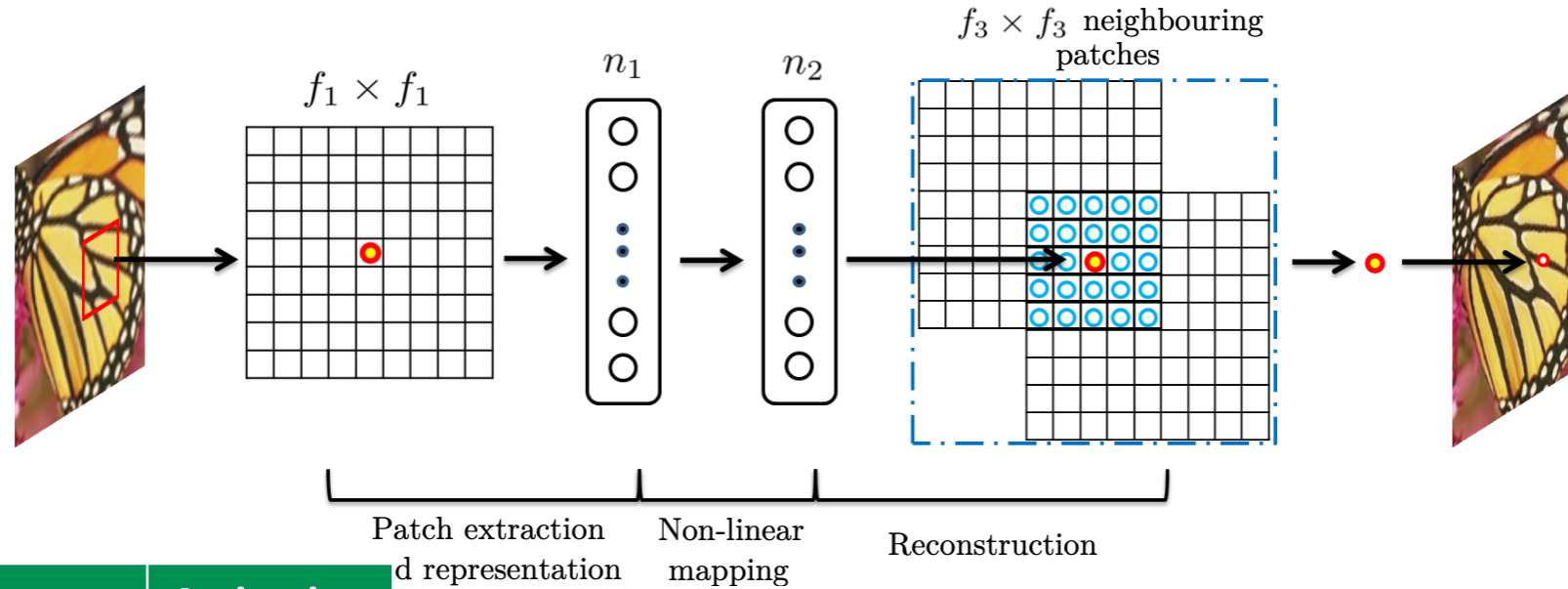
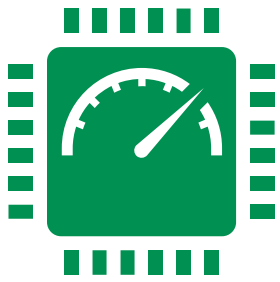
Increasing dataset sizes
⇒ training runtime

Model Size



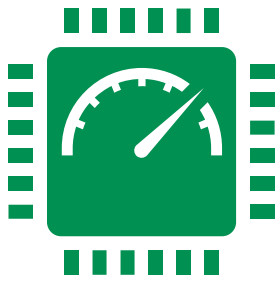
Increasing model sizes
⇒ inference runtime

SRCNN



Layer	Kernel	Features	Activation
0		1	
1	9 x 9	64	ReLU
2	1 x 1	32	ReLU
3	5 x 5	1	None

SRCNN has relatively simple architecture (not very deep!)



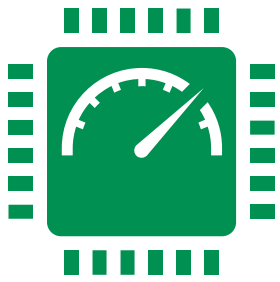
References

1. C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a deep convolutional network for image super-resolution," Computer Vision–ECCV 2014.
2. I. Goodfellow, Y. Benigo and A. Courville, Deep Learning, MIT Press., 2016
<http://www.deeplearningbook.org>
3. Vincent Dumoulin, Francesco Visin - [A guide to convolution arithmetic for deep learning](#)
<https://arxiv.org/abs/1603.07285>
4. A. Amidi and S. Amidi, "CNN Cheatsheet," <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>

Let's take a 5-minute Break



Example: Matrix Multiplication



Matrix Multiplication

Matrix product is fundamental operation in many numerical algorithms

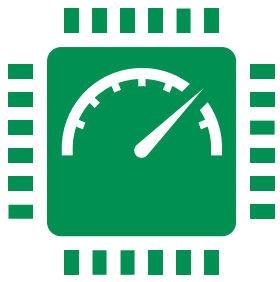
- Linear operation
- Multidimensional computation
- Opportunities for parallelization and management of memory usage

What is computational complexity of multiplying $n \times m$ and $m \times p$ matrix?

$\sim N \times M \times P$ multiply-accumulate operations (MACs)

Kastner et al, *Parallel Programming for FPGAs*.

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1p} \\ B_{21} & B_{22} & \cdots & B_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mp} \end{bmatrix}$$
$$\mathbf{AB} = \begin{bmatrix} (\mathbf{AB})_{11} & (\mathbf{AB})_{12} & \cdots & (\mathbf{AB})_{1p} \\ (\mathbf{AB})_{21} & (\mathbf{AB})_{22} & \cdots & (\mathbf{AB})_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ (\mathbf{AB})_{n1} & (\mathbf{AB})_{n2} & \cdots & (\mathbf{AB})_{np} \end{bmatrix}$$



Matrix Multiplication

```
// direct matrix multiplication
void matmul(int A[N][M], int B[M][P], int AB[N][P]) {
    ROW: for (int i = 0; i < N; i++) {
        COL: for (int j = 0; j < P; j++) {
            int ABij = 0;
            PROD: for (int k = 0; k < M; k++) {
                ABij += A[i][k] * B[k][j];
            }
            AB[i][j] = ABij;
        }
    }
}
```

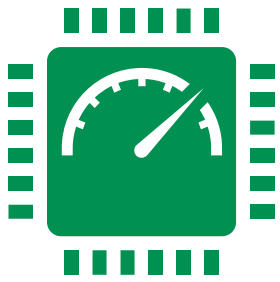
What options do we have in HLS to parallelize this computation?

Pipeline COL loop: $\sim N \cdot P$ cycles to complete
Pipeline ROW loop: $\sim N$ cycles to complete
Pipeline matmul: ~ 1 cycle
Partial unrolling at each point...

Kastner et al, *Parallel Programming for FPGAs*.

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1p} \\ B_{21} & B_{22} & \cdots & B_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mp} \end{bmatrix}$$
$$\mathbf{AB} = \begin{bmatrix} (\mathbf{AB})_{11} & (\mathbf{AB})_{12} & \cdots & (\mathbf{AB})_{1p} \\ (\mathbf{AB})_{21} & (\mathbf{AB})_{22} & \cdots & (\mathbf{AB})_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ (\mathbf{AB})_{n1} & (\mathbf{AB})_{n2} & \cdots & (\mathbf{AB})_{np} \end{bmatrix}$$

How does our choice determine our approach to memory layout?



Matrix Multiplication

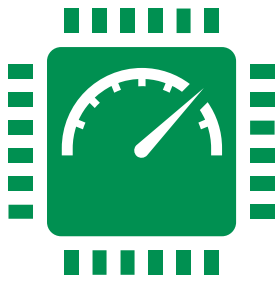
```
// direct matrix multiplication
void matmul(int A[N][M], int B[M][P], int AB[N][P]) {
    ROW: for (int i = 0; i < N; i++) {
        COL: for (int j = 0; j < P; j++) {
            int ABij = 0;
            PROD: for (int k = 0; k < M; k++) {
                ABij += A[i][k] * B[k][j];
            }
            AB[i][j] = ABij;
        }
    }
}
```

Kastner et al, *Parallel Programming for FPGAs*.

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1p} \\ B_{21} & B_{22} & \cdots & B_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mp} \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} (\mathbf{AB})_{11} & (\mathbf{AB})_{12} & \cdots & (\mathbf{AB})_{1p} \\ (\mathbf{AB})_{21} & (\mathbf{AB})_{22} & \cdots & (\mathbf{AB})_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ (\mathbf{AB})_{n1} & (\mathbf{AB})_{n2} & \cdots & (\mathbf{AB})_{np} \end{bmatrix}$$

How does our choice determine our approach to memory layout?

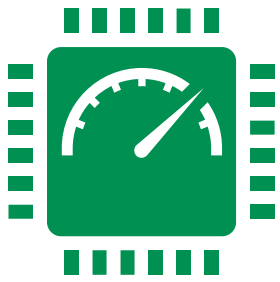


Block Matrix Multiplication

Blocking or *tiling* breaks computation of larger matrix down into computation of submatrices

- Find structure in algorithm, e.g., skip computation for blocks known a priori to be zero
- Reduce memory storage requirements and stream data to core as needed

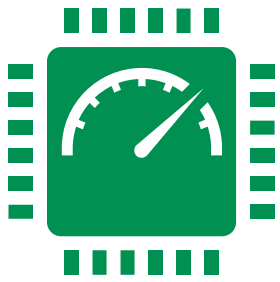
$$\begin{array}{|c|c|c|c|} \hline A_{11} & A_{12} & A_{13} & A_{14} \\ \hline A_{21} & A_{22} & A_{23} & A_{24} \\ \hline A_{31} & A_{32} & A_{33} & A_{34} \\ \hline A_{41} & A_{42} & A_{43} & A_{44} \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline B_{11} & B_{12} & B_{13} & B_{14} \\ \hline B_{21} & B_{22} & B_{23} & B_{24} \\ \hline B_{31} & B_{32} & B_{33} & B_{34} \\ \hline B_{41} & B_{42} & B_{43} & B_{44} \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline AB_{11} & AB_{12} & AB_{13} & AB_{14} \\ \hline AB_{21} & AB_{22} & AB_{23} & AB_{24} \\ \hline AB_{31} & AB_{32} & AB_{33} & AB_{34} \\ \hline AB_{41} & AB_{42} & AB_{43} & AB_{44} \\ \hline \end{array}$$



Block Matrix Multiplication

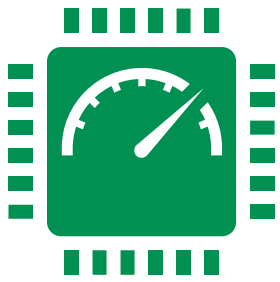
$$\begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} & B_{13} & B_{14} \\ B_{21} & B_{22} & B_{23} & B_{24} \\ B_{31} & B_{32} & B_{33} & B_{34} \\ B_{41} & B_{42} & B_{43} & B_{44} \end{bmatrix} = \begin{bmatrix} AB_{11} & AB_{12} & AB_{13} & AB_{14} \\ AB_{21} & AB_{22} & AB_{23} & AB_{24} \\ AB_{31} & AB_{32} & AB_{33} & AB_{34} \\ AB_{41} & AB_{42} & AB_{43} & AB_{44} \end{bmatrix}$$

Block Matrix Multiplication



$$\begin{array}{|c|c|c|c|} \hline A_{11} & A_{12} & A_{13} & A_{14} \\ \hline A_{21} & A_{22} & A_{23} & A_{24} \\ \hline A_{31} & A_{32} & A_{33} & A_{34} \\ \hline A_{41} & A_{42} & A_{43} & A_{44} \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline B_{11} & B_{12} & B_{13} & B_{14} \\ \hline B_{21} & B_{22} & B_{23} & B_{24} \\ \hline B_{31} & B_{32} & B_{33} & B_{34} \\ \hline B_{41} & B_{42} & B_{43} & B_{44} \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline AB_{11} & AB_{12} & AB_{13} & AB_{14} \\ \hline AB_{21} & AB_{22} & AB_{23} & AB_{24} \\ \hline AB_{31} & AB_{32} & AB_{33} & AB_{34} \\ \hline AB_{41} & AB_{42} & AB_{43} & AB_{44} \\ \hline \end{array}$$

Block Matrix Multiplication



$$\begin{array}{|c|c|c|c|} \hline A_{11} & A_{12} & A_{13} & A_{14} \\ \hline A_{21} & A_{22} & A_{23} & A_{24} \\ \hline A_{31} & A_{32} & A_{33} & A_{34} \\ \hline A_{41} & A_{42} & A_{43} & A_{44} \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline B_{11} & B_{12} & B_{13} & B_{14} \\ \hline B_{21} & B_{22} & B_{23} & B_{24} \\ \hline B_{31} & B_{32} & B_{33} & B_{34} \\ \hline B_{41} & B_{42} & B_{43} & B_{44} \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline AB_{11} & AB_{12} & AB_{13} & AB_{14} \\ \hline AB_{21} & AB_{22} & AB_{23} & AB_{24} \\ \hline AB_{31} & AB_{32} & AB_{33} & AB_{34} \\ \hline AB_{41} & AB_{42} & AB_{43} & AB_{44} \\ \hline \end{array}$$

```

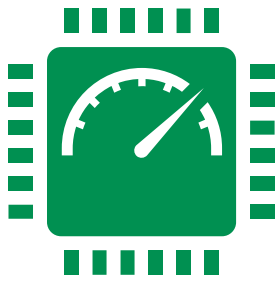
void blockmatmul(hls::stream<blockvec> &Arows,
                hls::stream<blockvec> &Bcols,
                blockmat &ABpartial, int it)
{
#pragma HLS DATAFLOW
    int counter = it % (SIZE/BLOCK SIZE);
    static DTYPE A[BLOCK SIZE][SIZE];

    if(counter == 0) { // only load the A rows when necessary
        loadA: for(int i = 0; i < SIZE; i++) {
            blockvec tempA = Arows.read();
            for(int j = 0; j < BLOCK SIZE; j++) {
#pragma HLS PIPELINE II=1
                A[j][i] = tempA.a[j];
            }
        }
    }

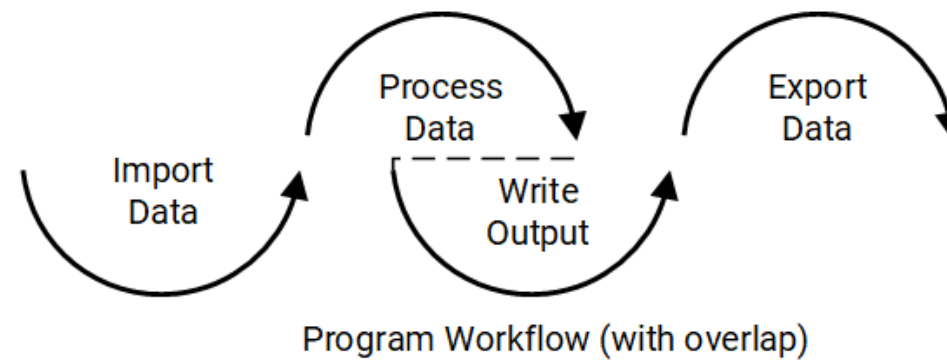
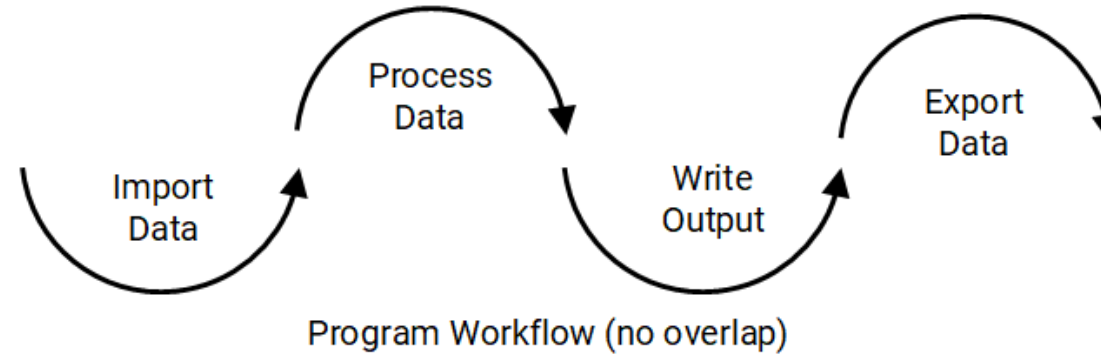
    DTYPE AB[BLOCK SIZE][BLOCK SIZE] = { 0 };
    partialsum: for(int k=0; k < SIZE; k++) {
        blockvec tempB = Bcols.read();
        for(int i = 0; i < BLOCK SIZE; i++) {
            for(int j = 0; j < BLOCK SIZE; j++) {
                AB[i][j] = AB[i][j] + A[i][k] * tempB.a[j];
            }
        }
    }

    // write output here ...
}

```

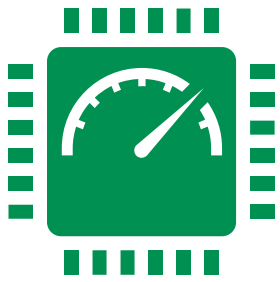


Producer-Consumer Model



X25607-073021

Dataflow



The HLS *Dataflow* model takes sequential functions and creates a task-level pipeline

#pragma HLS DATAFLOW

Tool handles inferring task-level parallelism and channels within region marked dataflow

- Creates synchronization signals to control execution between tasks
- User can manually specify whether channels are FIFO or PIPO

#pragma HLS STREAM type=<T> variable=<V> depth=<D>

```
void diamond(data_t vecIn[N], data_t vecOut[N])
{
    data_t c1[N], c2[N], c3[N], c4[N];
    #pragma HLS dataflow
    funcA(vecIn, c1, c2);
    funcB(c1, c3);
    funcC(c2, c4);
    funcD(c3, c4, vecOut);
}
```